ENORM: A Framework For Edge NOde Resource Management

Nan Wang, Blesson Varghese, Michail Matthaiou, and Dimitrios S. Nikolopoulos

Abstract—Current computing techniques using the cloud as a centralised server will become untenable as billions of devices get connected to the Internet. This raises the need for fog computing, which leverages computing at the edge of the network on nodes, such as routers, base stations and switches, along with the cloud. However, to realise fog computing the challenge of managing edge nodes will need to be addressed. This paper is motivated to address the resource management challenge. We develop the first framework to manage edge nodes, namely the *E*dge *NO*de *R*esource *M*anagement (ENORM) framework. Mechanisms for provisioning and auto-scaling edge node resources are proposed. The feasibility of the framework is demonstrated on a PokéMon Go-like online game use-case. The benefits of using ENORM are observed by reduced application latency between 20-80 percent and reduced data transfer and communication frequency between the edge node and the cloud by up to 95 percent. These results highlight the potential of fog computing for improving the quality of service and experience.

Index Terms—Fog computing, edge nodes, resource management, provisioning, scaling resources

1 Introduction

Currently, applications that are hosted on cloud servers, typically provide services from a cloud data center as shown in Fig. 1. Some of these servers are replicated across multiple data centers located in different geographical regions for reducing the workload and improving the Quality-of-Service (QoS). However, this will not be suitable in the future given that over 25 billion devices are estimated to be added to the network by 2020.[1] The QoS will degrade as more devices need to be catered for by existing communication and computing infrastructure [1]. Three problems will need to be addressed to tackle this. They are minimising communication latency between users and computing devices and reducing both data traffic to the cloud and communication frequency between devices and the cloud.

In this paper, we prototype a *'Fog Computing'* [2], [3] framework to bring computing towards the edge of the network, which is closer to the user devices, so that the latency of communication, data traffic to the cloud and frequency of communication between user devices and the cloud can be reduced. This is shown in Fig. 2. In contrast to the two-tier cloud computing architecture, in which a user interacts only with the cloud, the fog computing architecture is at least a three-tier architecture. An additional edge node (for example, mobile base stations, routers, and switches) layer is inserted between the user and the cloud.

1. http://www.gartner.com/newsroom/id/2636073

• *The authors are with Queen's University Belfast, Belfast BT7 1NN, United Kingdom.*
E-mail: {nwang03, b.varghese, m.matthaiou, d.nikolopoulos}@qub.ac.uk.

Manuscript received 5 Jan. 2017; revised 31 Aug. 2017; accepted 12 Sept. 2017. Date of publication 18 Sept. 2017; date of current version 8 Dec. 2020. (Corresponding author: Nan Wang.)
Digital Object Identifier no. 10.1109/TSC.2017.2753775

Resource management at the edge of the network will play an important role in realising fog computing. However, this is challenging in a number of ways. We have identified three problems that will need to be solved in this space.

The first problem is provisioning edge nodes for executing workloads offloaded from the cloud. While there exists a marketplace for executing workloads on the cloud, such a marketplace is unavailable for edge nodes. This makes it challenging to provision an edge node since directives and standard protocols for initialising services on a potential edge node and for communicating between the cloud and edge nodes are not fully known and developed.

The second problem is deploying workloads on edge nodes. The following two important questions must be considered: (i) How to deploy a workload on the edge node? On the cloud, Virtual Machines (VMs) are usually employed to execute a workload. However, VMs are less likely to be suitable on edge nodes given the availability of limited hardware resources. (ii) How much of the workload can be deployed on the edge node? Large workloads are easily executed on the cloud given the access to a large amount of resources. It is challenging to decide appropriate workloads to execute on an edge node given its limited hardware resources and varying computation availability introduced by its primary service. For example, the primary service of a Wi-Fi access point is to route Internet or mobile traffic, which we refer to as the*basic service*.

The third problem is managing resources on edge nodes. Due to its basic service, workloads offloaded to an edge node are of secondary priority, since the primary service cannot be compromised. This makes resource management on edge nodes challenging because the resources allocated for the workload will have to dynamically scale (or auto-scale) given that there are limited hardware resources available on the edge node when compared to a cloud server.

1939-1374 © 2017 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: International Institute of Information Technology-Raipur. Downloaded on January 30,2026 at 10:44:11 UTC from IEEE Xplore. Restrictions apply.

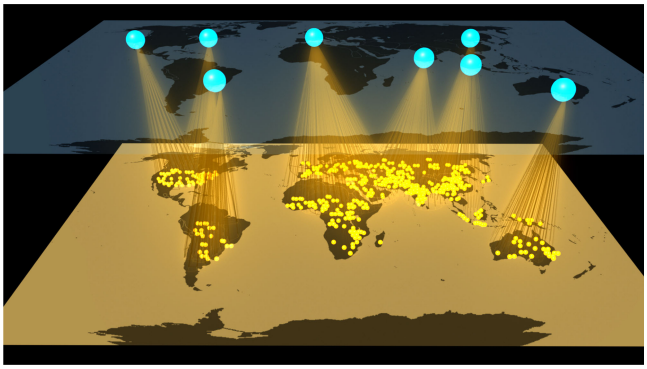


Fig. 1. Existing computing model that employs the cloud. Blue dots are example locations of cloud data centers and yellow dots show devices that interact with servers in the data center.

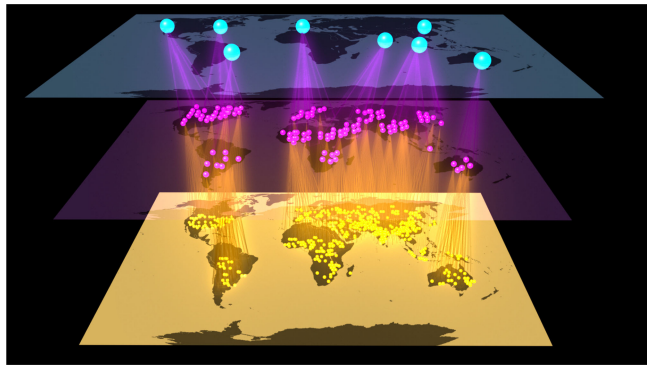


Fig. 2. The fog computing model. While the blue dots show example locations of cloud data centers, the purple dots highlight the larger volume of edge nodes that are present in the network. The user devices communicate more frequently with the edge nodes, which in turn interact less frequently with the cloud servers.

Additionally, the allocation of resources to host multiple tenants will need to be considered.

Existing resource management frameworks in distributed computing are suitable in the context of clusters [4] and clouds [5], but do not consider the edge of the network. In this paper, we aim to tackle the above three problems by proposing and developing ENORM, an *E*dge *NO*de *R*esource *M*anagement framework that integrates the edge of the network in the computing ecosystem to realise fog computing. We first propose a novel provisioning and deployment mechanism to integrate an edge node with a cloud server. Contrary to cloud-based deployments, our technique is developed for resource limited environments. Consequently, our mechanism only requires simple implementation. Second, an auto-scaling mechanism to dynamically manage edge resources is proposed. This mechanism is essential to meet service objectives for improving the QoS and to safely use edge nodes without affecting other workloads that are executed on them.

The feasibility of ENORM is validated on a PokéMon Go-like location-aware and latency-sensitive online game use-case. The key result is that there are significant benefits in improving the QoS of a large number of users in a given location by employing ENORM for achieving the fog computing based use-case. The low overheads of the framework not only indicate that fog computing is feasible, but also demonstrate the lightweight nature of the mechanisms developed within ENORM. When compared to a cloud-only model, the application latency is reduced between 20-80 percent. Similarly, the data traffic and the communication frequency between the edge node and the cloud server are both reduced by up to 95 percent. All these results showcase the benefit of integrating the edge in the computing ecosystem with the cloud in fog computing.

The research contributions of this paper are first the integration of the edge of the network in the computing ecosystem through the development of a provisioning mechanism that facilitates communication between edge nodes and the cloud. Currently, there are no distributed computing frameworks that can manage edge node resources in the computing ecosystem. ENORM on the other hand can provision edge nodes for offloaded workloads from the cloud. Second, ENORM incorporates a low overhead and dynamic auto-scaling mechanism to add or remove resources for efficiently handling the workloads on the edge node. We note that using ENORM over 16,000 users can be serviced by the edge node by only incurring an additional 5-second overhead for auto-scaling.

The remainder of this paper is organised as follows. Section 2 presents ENORM for managing resources of edge nodes. Section 3 considers provisioning and auto-scaling in ENORM. Section 4 demonstrates an online game use-case employed to test the feasibility of fog computing using ENORM. Section 5 evaluates the overheads and performance gain of using ENORM. Section 6 presents related work. Section 7 concludes this paper.

2 The ENORM Framework

To solve the three challenging problems presented before in realising fog computing, we propose a resource management framework, referred to as '**ENORM**: **E**dge **NO**de **R**esource **M**anagement'. The architecture we propose for ENORM works across three-tiers as shown in Fig. 3. The *top tier* is the cloud tier, where application servers are hosted. In the cloud-only execution model, connections from user devices are established with the servers located in one or more data centers.

To enable the use of the cloud in conjunction with edge nodes, our framework deploys a *cloud server manager* on each application server. This server (i) communicates with potential edge nodes requesting computing services, and (ii) deploys partitioned servers on the edge node, and (iii) receives updates from the edge node to update the global view of the application server on the cloud.

The *bottom tier* is the user device tier. Multiple devices, such as smartphones, wearables and gadgets, are connected to the cloud application servers. In the cloud-only execution model, the devices connect to the application servers through traffic routing nodes. The basic services of the mobile base stations in the cellular network or routers in the wireless local network are used to communicate to the centralised cloud server. However, using ENORM, the computing capabilities of the edge nodes are made use of.

The *middle tier* is the edge node tier. This could be multiple layers of edge nodes that scale both horizontally and vertically. In this paper, the focus is on a single edge node (this is the first framework for fog computing resource

Authorized licensed use limited to: International Institute of Information Technology-Raipur. Downloaded on January 30,2026 at 10:44:11 UTC from IEEE Xplore. Restrictions apply.

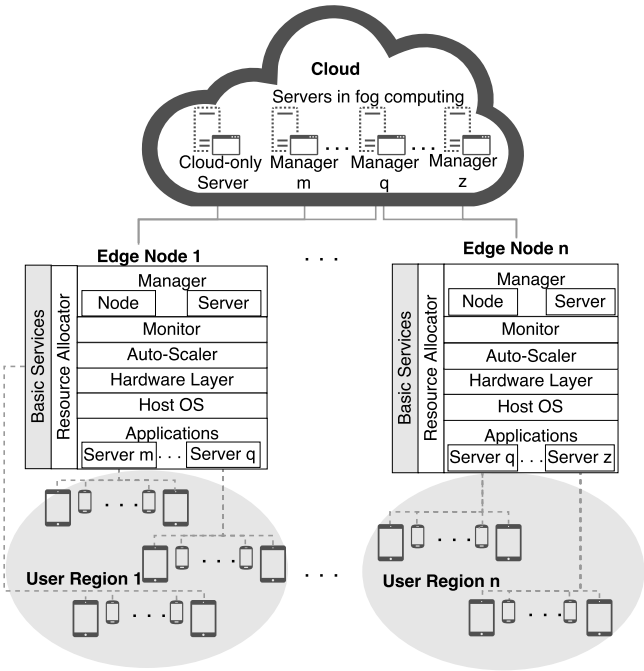


Fig. 3. Architecture of ENORM in the fog computing ecosystem. Typically, using the cloud-only model user devices connect to a cloud server through the basic service offered by a traffic routing node. In ENORM, in addition to the basic services, user devices are serviced by the edge nodes that host servers offloaded from cloud servers.

management and multiple nodes will be considered in the future). As seen in Fig. 3, the servers making use of the edge communicate with on-demand edge servers to support users in regions covered by each edge node. For example, users start an application (App q, which is hosted by Server q) in multiple regions covered by edge nodes (Edge Node 1 and n). Connections are established with the respective application servers on the cloud. A partitioned application server of App q is deployed to the edge nodes (the details of how the edge node is provisioned and connections are established will be presented in Section 3).

The partitioned server on the edge node is different from the cloud application server in that localised data relevant to the users covered by the edge node is maintained. The global view is maintained on the cloud server and the edge node updates the cloud server. When edge nodes cannot provide computing services (for example, if the edge node is overloaded during peak hours and there are no spare cycles) or the edge nodes cannot improve the QoS of the application, then the deployed edge server will be terminated and users connect to the cloud application server as in the cloud-only execution model.

Our proposed ENORM framework, uses the following five components on the edge node. They are:

(i) *Resource Allocator*: In providing computing as a service, the basic service of an edge node cannot be compromised. This involves prioritising the basic service over any offloaded workloads from cloud servers. It is therefore essential to know the free resources that are available on the edge node. The resource allocator keeps track of the available CPU cores and memory.

(ii) *Edge Manager*: The Edge Manager comprises two subcomponents, namely the Node Manager and the Server Manager. The node manager deals with the requests that are obtained by the server manager from a cloud server. When a request is made from the cloud manager, a decision on whether to accept the request is made by the node manager (this decision is based on whether there are free resources on the edge node and whether the priority of the requesting application is higher than or equal to that of an executing server on the edge). The response is sent back to the requesting cloud server. Once a request is accepted, the server manager initialises a container, allocates necessary ports for communicating and updates firewall settings.

In our framework, Linux containers are employed to provide applications isolation on the edge node through virtualisation. Our rationale for choosing container virtualisation technology instead of VMs is due to the reduced boot up times and enhanced isolation provided by the former [6]. Moreover, on limited hardware platforms, such as edge nodes, containers are appropriate given that they are relatively lightweight and have low overheads.

(iii) *Monitor*: A number of metrics related to each application edge server is monitored periodically. Our monitor tracks communication latency (using standard commands such as ping and assuming that the user devices have static IP addresses) and computing latency (obtained from timestamps on server logs). These metrics are employed by the auto-scaler on whether more resources need to be allocated or existing resources need to be removed from an application server. For example, the edge server performance is monitored in terms of round-trip application latency and hardware utilisation of CPU and memory.

(iv) *Auto-scaler*: Based on the metrics obtained from resource allocator and monitor, the auto-scaler dynamically allocates/de-allocates hardware resources to the containers executing application servers (considered in Section 3). Dynamic allocation of resources is required to (i) ensure that the basic service has sufficient resources such that there is no overload due to additional application servers, and (ii) modify allocated resources to accommodate more users or new application servers on the edge node. If an application server cannot obtain resources or even if obtained could not improve performance, then the edge manager terminates the edge server. This is considered further in Section 3.2.

(v) *Application Edge Server*: The partitioned server from the cloud is hosted on the edge node. This server interacts with user devices before forwarding data to the cloud.

3 Resource Management in ENORM

In this section, we present provisioning and auto-scaling that are essential to ENORM. Provisioning in our framework enables cloud servers to offload workloads on to edge nodes. Auto-scaling takes resource availability on the edge node into account and allocates/de-allocates resources provided to a workload. Table 1 shows the mathematical notation we have used in this paper.

3.1 Provisioning

Procedure 1 shows the provisioning mechanism of ENORM on an edge node in three stages. They include handshaking, deployment and termination. When an edge manager starts, it is initialised by checking whether the node can support

Authorized licensed use limited to: International Institute of Information Technology-Raipur. Downloaded on January 30,2026 at 10:44:11 UTC from IEEE Xplore. Restrictions apply.

TABLE 1
Notation Used in the Proposed Resource Management Mechanisms

Parameter	Description	Source
$service$	Flag on the availability of the services on an edge node	Edge manager
S	A set of n edge servers hosted on n containers in an edge node, ordered by Pri. $s_i \in S, i = 1, ..., n$	
r^u	One unit of CPU and memory	
Prt	A list of all available ports on an edge node	
$termType$	Flag for 'single' or 'multiple' termination of containers on an edge node	
R	Free CPU and memory on the edge node available for S	Resource allocator
Pri_i	Priority of s_i	Cloud manager
$cldTerm_i$	Flag for overriding decision of the edge manager on s_i by the cloud for termination	
U_i	A set of users to connect to s_i	
l^i	The desired application latency objective of s_i	
Prt_i	A list of ports used by s_i	
$request_i$	A list of $[s_i, Pri_i, Prt_i, l_i, U_i]$ to request edge services on an edge node	
l_i^n	Measured average round-trip network latency of s_i	Monitor
l_i^c	Measured average computing latency of s_i	
l_i^a	Computed round-trip application latency of s_i; $l_i^a = l_i^n + l_i^c$	
r_i	CPU and memory used by s_i on the edge node; r_n is the CPU and memory used by s_n on the edge node	
r^r	CPU and memory released by terminating servers on the edge node	
$decision$	Flag for 'scaleup' or 'scaledown' containers on an edge node	Auto-scaler

edge services (Line 1). If positive, then a handshaking protocol is initiated (Line 2). Additionally, the cloud manager can terminate its edge server (Lines 4-5).

Procedure 1. Provisioning mechanism

Data: $service, request_i, cldTerm_i, S, R, r^u, termType_i$
1: **while** $service == true$ **do**
2: Handshake($request_i, S, r^u, R$);
3: Deploy($s_i, U_i, S, R, r^u, termType_i, cldTerm_i$);
4: **if** $cldTerm_i == true$ **then**
5: Terminate($S, s_i, single, true$);
6: **end**
7: **end**

Handshaking: Procedure 2 presents the handshaking protocol. The edge node listens for incoming requests from cloud managers. If the edge node can provide services to a requesting cloud manager, then the cloud manager is notified and returns an edge server setup request. This request includes the name of the application, priority level, ports required to connect to user devices, users to be served by the edge server and latency objective (provided as $request_i$ by the cloud manager to the edge manager in Procedure 1). In ENORM, handshaking is not only identifying an edge node for deploying an application, but also initialising a container on the edge node and setting up appropriate firewalls.

A resource check and a priority check is performed when the setup request is received (basic service takes highest priority and other workloads with a high priority can be executed). If the new request ranked lower than any of the currently executing edge servers or there is not sufficient resource to launch a new container, then the request is rejected (Line 10). The cloud manager may then decide to request services from another edge node. If the request passes the resource and priority checks (Line 1), the edge manager further checks whether the proposed ports are available for supporting communication between the edge server and the user devices (Lines 2-4). An access port is generated (Line 5) for remote access from the cloud server manager. When unique ports are successfully allocated to service the current request, an Operating System (OS) container is launched with a default image offered by the edge manager (Line 6), which will be used for executing the workload offloaded by the cloud server manager. The container is allocated a default amount of resources (for example, one core and 200 MB of RAM). After the container is booted on the edge node, the edge manager configures this container (Line 7), such that the application executed on this container will be visible for connecting user devices as well as for managing by the cloud server. This is done through port forwarding using*iptables* command[2]. After configuring the container, a message is sent to the cloud server manager (Line 8).

Procedure 2. Handshaking mechanism

Data: $request_i, S, r^u, R$
1: **if** $i <= n$ & $R >= r^u$ **then**
// S is ordered by Pri
2: **if** $Prt_i \notin Prt$ **then**
3: assign the same no. of ports from Prt to Prt_i;
4: **end**
5: assign an additional port from Prt to Prt_i;
6: initialise LXC container on s_i, Prt_i;
7: update the firewall of Prt_i;
// using iptables command
8: send response to cloud server manager;
9: **else**
10: reject($request_i$);
11: **end**

Deployment: Procedure 3 shows deployment on the edge nodes. The cloud server manager deploys a partitioned edge server application and installs software packages on the container required by the server remotely (Lines 1-2).

2. https://linux.die.net/man/8/iptables

Authorized licensed use limited to: International Institute of Information Technology-Raipur. Downloaded on January 30,2026 at 10:44:11 UTC from IEEE Xplore. Restrictions apply.

The container is customised for each application server by the cloud server manager, which in turn avoids the wastage of resources if a generic container with comprehensive packages of libraries is booted. Once the application server is launched on the edge node (Line 3) it can start receiving requests from user devices. The cloud server manager redirects the application users covered by the edge node through a configuration file that points to the edge server instead of the cloud server (Line 4). Once users are connected the resources allocated to the container can be scaled (more resource can be allocated if available or de-allocate resources if they are not required; Line 5), which will be considered in the the next section.

Procedure 3. Deployment mechanism

Data: s_i, U_i, S, R, r^u
1: install software packages in s_i;
2: deploy partitioned server image in s_i;
3: launch s_i;
4: redirect U_i to s_i;
5: $autoScale(S, s_i, R, U_i, r^u, termType_i, cldTerm_i)$;

Termination: In the fog computing model, it is anticipated that when compared to cloud servers, edge servers will be used for shorter time intervals given the demand and limited resources on edge nodes. Therefore, in addition to auto-scaling, the edge manager will need to decide when an application server on the edge needs to be removed from the edge node as shown in Procedure 4. The edge manager terminates an edge service in the following three cases, which are considered in the auto-scaling mechanism. First, there are no free resources to support the edge service. Second, the edge service is not required any more (the edge server has been idle for a time period). Third, the edge service does not improve the QoS of the application (the performance constraints cannot be satisfied by an edge server deployment). The cloud server manager can override the edge server for terminating its edge service, so that when no budget is available for additional edge service usage, the cloud server manager withdraws its server from the edge.

Procedure 4. Termination mechanism

Data: $S, s_i, termType, cldTerm_i$
1: **if** $cldTerm_i == true \text{Or} termType == single$ **then**
2: migrate and redirect U_i to the cloud;
3: stop and destroy LXC container hosting s_i;
4: $S = S - \{s_i\}$;
5: **end**
6: **if** $cldTerm_i == false$ ***Or*** $termType == multiple$ **then**
7: **for** $\forall s_i \in [s_i, s_n]$ **do**
8: migrate and redirect U_i to the cloud;
9: stop and destroy LXC container hosting s_i;
10: $S = S - \{s_i\}$;
11: **end**
12: **end**

A single or multiple containers can be terminated (based on the value of $termType_i$). When an edge application server is terminated, the associated data containing local updates is migrated to the cloud (the local data will be appended to the global data maintained by the cloud server manager; Lines 2 and 8). This is realised through a key-value based data store, Redis,[3] which supports data migration between two servers. The user devices affected will have to be redirected back to the cloud server until the next edge node can be provisioned by the cloud server manager.

3.2 Auto-scaling

The importance of scaling resources allocated to an edge application server is in that (i) the edge nodes have limited hardware resources (will need to be primarily used for basic services) and (ii) the application server executing on the node requires more or less resources to the meet the QoS objective (in this paper, we only consider application latency). Procedure 5 shows the auto-scaling mechanism we have proposed in this paper. This is not a one time method, but happens periodically (in this paper, every 5 minutes) during the execution of an application server on the edge node. The edge manager, resource allocator and monitor are required to enable auto-scaling.

Procedure 5. Auto-scaling mechanism

Data: $S, s_i, R, U_i, r^u, termType_i, cldTerm_i$
1: **for** $\forall s_i \in S$ **do**
`// S is ordered by` Pri
2: $l_i^n = ping(U_i)$;
3: measure l_i^c;
4: $l_i^a = l_i^n + l_i^c$;
5: **if** $R >= r^u$ **then**
6: **if** $U_i! = \emptyset$ ***Or*** $l_i^n < l^i$ **then**
7: **if** $l_i^a > l^i$ **then**
8: $scale(s_i, scaleup, r^u, S, R)$;
9: **else**
10: $scale(s_i, scaledown, r^u, S, R)$;
11: **end**
12: **else**
13: $terminate(S, s_i, single, false)$;
14: **end**
15: **else**
16: $terminate(S, s_i, multiple, false)$;
17: **end**
18: **end**

A list of application servers executing on the edge node and their priority levels is maintained by the edge manager (Line 1). The priority of an application is set by the cloud manager that owns the application. In this paper, we only consider a static priority. The priority of the application does not change during execution. If the priority needs to change, then the cloud manager will need to relaunch the application on the edge with a new priority. The application with the highest priority is first considered by the auto-scaler. The network latency is measured by pinging users from the server (Line 2; the reason for choosing pinging is because our demonstrator use case transmits a relatively small amount of data in each HTTP request. However, for other applications, varying data sizes may be more suitable). It is then checked if the amount of free resources available on the edge node reported by the resource allocator is

3. https://redis.io/

Authorized licensed use limited to: International Institute of Information Technology-Raipur. Downloaded on January 30,2026 at 10:44:11 UTC from IEEE Xplore. Restrictions apply.

larger than the predefined minimum amount of resources required by the edge application server (Line 5). If this resource check fails, then the high priority server along with all other servers that ranked below it will be migrated back to the cloud from where they were offloaded (Line 16). The terminate mechanism used here is the same as presented in Procedure 4. This is done to ensure the basic services on the edge nodes retain the highest priority. If the resource check passes, then the auto-scaler further checks if there is a need for the edge application server on the node (whether users are covered by this edge node or whether the latency of the application can be reduced on the edge node; Line 6). The existence of users (whether users are connected to the edge server) and network latency are considered to decide whether the edge server can deliver the desired improvement or if migrating the application server elsewhere to the cloud or to another edge node can be of more benefit (Line 13). The computed application latency is compared with the latency objective of the cloud server manager in its service request (Line 7). If the application latency is higher than the service objective, (or the edge server has not been performing as expected), then the container hosting the edge server will be allocated more resources (Line 8). Resources are removed from the container when the latency is less than the objective (Line 10).

Procedure 6. Scaling Mechanism

Data: $r^u, S, decision, R, s_i$
1: **if** $decision == scaleup$ **then**
2: Measure r_i;
// using LXC command $lxc - cgroup$
3: **if** $R >= r^u$ **then**
4: $r_i = r_i + r^u$;
5: **else**
6: $r^r = 0$;
7: **while** $R < r^u$ **do**
8: Measure r_n;
9: terminate($s_n, S, single, false$);
10: $R = R + r_n$;
11: $r^r = r^r + r_n$;
12: $n = n - 1$;
13: **if** $R >= r^u$ ***Or*** $n == i$ **then**
14: **break** ;
15: **end**
16: $r_i = r_i + r^r$;
17: **end**
18: **end**
19: **end**
20: **if** $decision == scaledown$ **then**
21: Measure r_i;
22: $r_i = r_i - r^u$;
23: **end**

When an application server is decided to be scaled, the scaling mechanism as shown in Procedure 6 first checks the decision on either "scaleup" or "scaledown" (Line 1 and 20). To scale up, i.e., to allocate more resources, the edge monitor first checks if there are additional available resources R on the edge node to support this (Line 3). If there are resources, then one more resource unit r^u (one core of CPU and 200MB of RAM in this paper) is added to the container (Line 4). Re-configuring the resource limits of a container is realised through $cgroup$, i.e., control group, which is a feature of Linux kernel that limits, accounts for and isolates the resource usage (for example, CPU and memory) of a container. If the available resource is not enough to support scaling up, then the container with lowest priority in the container set S will be terminated to release its resources so that there is more available resources R (Line 9). The containers with lower priorities will be terminated one by one until R is sufficient to support scale up or there are no more containers with lower priorities (Line 13). To scale down, a unit of resources is removed from the current resources that server s_i is allocated (Line 22). At the end of the scaling process, the application edge server s_i is updated with the new quota of resources (Lines 4, 16 and 22). When an application server is either scaled or migrated, the auto-scaler takes the next server from the list to repeat the process above until the resource check fails on an edge node.

Auto-scaling approaches can introduce instability when the edge node resources are exhausted and if in an auto-scaling round containers with lower priority were scaled down. This can be mitigated if the amount of resources that need to be removed from a container are known beforehand. Our algorithm does not introduce this instability since we do not progressively scale down containers with lower priority. Instead the container with the lowest priority is terminated until there is sufficient resources for a container with the highest priority to scale up.

4 An Online Game Use-Case

In this paper, we choose a location-aware online game use-case for testing the feasibility of the ENORM framework. Such games are naturally a good fit in the context of fog computing. For location-aware games the server maintains a global view of connected users. However, this server can be partitioned such that local views specific to a location are managed by edge nodes. The global view will be updated periodically, but less frequently, on the cloud server. Frequent location specific changes are updated locally.

The online-game chosen is an open-source implementation of a game similar to Pokémon Go,[4] named iPokeMon.[5] iPokeMon comprises a client for the iOS platform, which can be used on mobile devices, and a server that is hosted on a public cloud. It maps virtual reality on to real world in which users can walk through streets to discover, fight against wild virtual creatures, named Pokémons, that are geographically distributed. This is latency sensitive since the virtual environment of the user needs to be frequently updated as a user navigates in a location. If the application is serviced from a distant cloud data center, then user experience is affected due to lags in refreshing the environment.

The Pokémon Go server is known to have crashed multiple times during its launch due to extreme user activity.[6,7] For such a game not only is it essential to replicate servers

4. http://www.pokemongo.com/
5. https://github.com/Kjuly/iPokeMon
6. http://www.forbes.com/sites/davidthier/2016/07/07/pokemon-go-servers-seem-to-be-struggling/#588a88b64958
7. https://www.theguardian.com/technology/2016/jul/12/pokemon-go-australian-users-report-server-problems-due-to-high-demand

Authorized licensed use limited to: International Institute of Information Technology-Raipur. Downloaded on January 30,2026 at 10:44:11 UTC from IEEE Xplore. Restrictions apply.

in different data centers, but given the large number of user connections, an edge layer between the cloud and the user can reduce the distance of communication.

We designed the iPokeMon game server to be hosted on an Amazon Web Services Elastic Compute Cloud[8] VM. The server was hosted on a t2.micro[9] VM in the Amazon Dublin data center, which is geographically closest to the authors location in Belfast (an additional server was hosted in N. Virginia for the purpose of comparison).

4.1 Implementing the Fog Computing Based Game Using ENORM

There are three requirements for implementing the above cloud-only iPokeMon as a *'fog computing based'* game. First, the game server needs to be partitioned such that the global view of the system is maintained on the cloud server and the local view on the edge node. In our model, the server is manually partitioned at function level. Functions related to users and location data are used to create the edge server. Only users existing on the cloud server are directed to edge nodes. The functionality to create and authenticate new users resides on the cloud server. The partitioned server residing on the edge node generates location data, such as region code since a user may traverse through new regions that is not associated with the user on the cloud. The edge node then updates this information in the global map, which is used to annotate places in the application client.

Second, iPokeMon needs to be modified so that it can connect to an IP address dynamically. The IP address of iPokeMon server is defined in a configuration file maintained by the cloud manager. By default it points to the cloud server, therefore when a user starts to play iPokeMon, the device is connected to the cloud server. Once an edge server is available, the cloud manager updates the configuration file with the IP address of the edge server. When a new user request is created, iPokeMon will switch connection to the edge server according to the configuration file. When the edge server terminates, the user is directed back to the cloud server.

The third requirement is partitioning the iPokeMon database at run time to support the migration of user-specific data from the cloud server to the edge node. User data is maintained in a Redis database on the cloud using predefined naming standards. Each user has a number of keys in the database which can be filtered. The keys and values related to all users that will connect to a specific edge node are copied to the edge server during deployment. Similarly, when the edge server is terminated, the updated user data on the edge is migrated to the cloud server and merged with the database containing the global view.

For executing the fog computing based iPokeMon game, data will be sent from a user device to the game server through a traffic routing node, such as a mobile base station. We used an ODROID-XU board[10] to represent the computing resources of a small cell base station. The board has one ARM Big.LITTLE architecture Exynos 5 Octa processor and 2 GB of DRAM memory. The processor runs Ubuntu 14.04 LTS. This Odroid board served as the edge node, which was

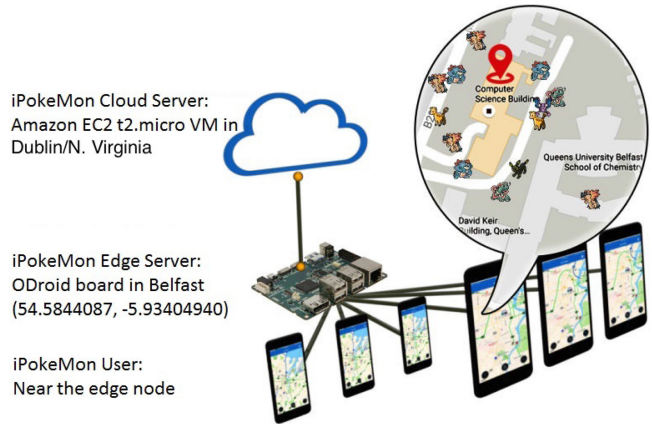


Fig. 4. The implementation of fog computing based iPokeMon game using the ENORM framework. The cloud server is on Amazon EC2 and the edge server is on an Odroid board that connects user devices.

located in the Computer Science Building of the Queen's University Belfast in Northern Ireland.

Fig. 4 shows our implementation of the fog computing based iPokeMon game using ENORM. The user creation and verification requests are made to the cloud server, after which the cloud manager makes a request for computing services to a potential edge node. Following this handshaking described in Section 3.1 is established. If this request is accepted by the edge node, then the edge manager initialises a container for the iPokeMon edge server. The cloud manager deploys the iPokeMon edge server and clones the data (to the edge database) of the users that will be connected to the edge node.

User data rapidly changes when the game is played. For example, the GPS coordinates of the player and the Pokémons. The local view on the edge server is updated by frequent update requests that are sent to the edge server. When the edge server has to be terminated, as considered in Section 3.1, then the edge database is merged with the global database located on the cloud. The user is redirected back to the cloud server for continuing the session. If a new edge node is available, then the above process is repeated.

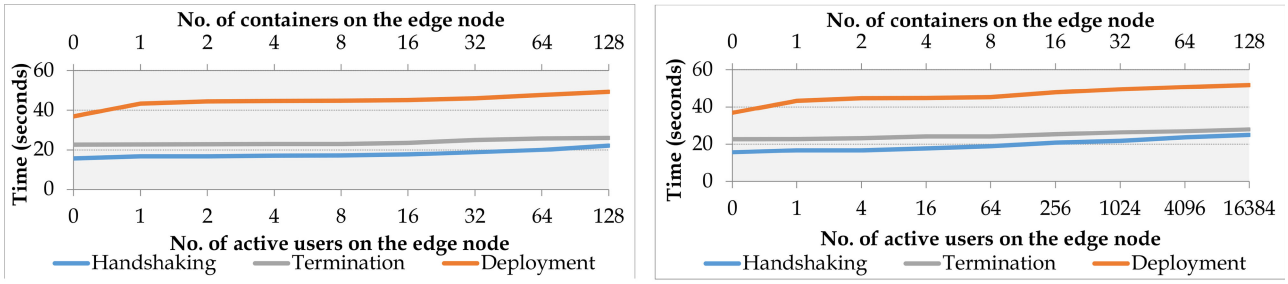
5 Experimental Studies

In this section, we evaluate our fog computing based implementation of the iPokeMon game. The partitioned game servers on the edge node were stress tested using Apache JMeter[11]. One session of a connection (the user is playing the iPokeMon game) between the user device and the edge server hosted on a container is recorded for 20 minutes. During this time the number and types of requests and the parameters sent through the requests are recorded. Subsequently, JMeter stress tests single and multiple edge servers by creating virtual users and sending requests to the edge server(s) from the virtual users in the experiments.

The activity of 'N' virtual users is defined by considering two types of user behaviour. First, aggressive user behaviour, in which the activity of each user is considered by randomly selecting only data intensive requests with no pauses between these requests for a 5-minute period from the pre-recorded 20 minutes. For example, when a user is

8. https://aws.amazon.com/ec2/
9. https://aws.amazon.com/ec2/instance-types/
10. http://www.hardkernel.com/
11. http://jmeter.apache.org/

Authorized licensed use limited to: International Institute of Information Technology-Raipur. Downloaded on January 30,2026 at 10:44:11 UTC from IEEE Xplore. Restrictions apply.



(a) For a single user on each container

(b) For multiple users on each container

Fig. 5. Overheads in provisioning edge servers using ENORM on an Amazon EC2 server located in the Dublin data center.

continuously playing by rapidly tapping the game screen with few breaks. All moves and taps made by the user will need to be transmitted from the phone to the game server. This is highly unrealistic, but is employed to represent a bandwidth-hungry task. Second, mixed user behaviour, in which the activity of each user is considered by equally selecting both data intensive requests (with pauses) and regular requests for a 5-minute period from the pre-recorded 20 minutes. For example, when a user plays iPokeMon strategically, by taking time for a next move before tapping the game screen. Some moves may require a very small amount of data to be transmitted while others may require larger amounts of data. This ensures different behaviours of the virtual users and captures a real world setting in which there are a combination of aggressive and passive users.

For comparing the cloud-only and fog computing based implementation using ENORM we have stress tested the cloud server. Each experiment shown in this paper is based on the stress test for 5 minutes and represents an average of 5 executions. To simulate the effect of a dynamic basic service (utilisation of resources on the edge node changes) on the free resources R, an open dataset[12] containing 142 days of mobile phone records was used to extract hourly patterns. We considered bursty patterns of basic services representative of variable resources utilisation on edge nodes.

Our experimental evaluation demonstrates the following: (i) the overheads in provisioning and auto-scaling using ENORM for the iPokeMon game, (ii) the improvement in the QoS when using ENORM measured by (a) user perceived latency of communication between the device and the edge node, (b) the data transfers between the user device and the cloud server, and (c) the communication frequency between the user device and the cloud server.

5.1 Overheads

We initially determined the overhead in setting up edge server using ENORM. This provides us with an insight into the expense incurred for using edge server as well as determining the impact of bringing about changes, such as auto-scaling, to the edge server, in terms of time. Here we explore the overheads for provisioning and auto-scaling.

Provisioning: Figs. 5 and 6 show the overhead in provisioning iPokeMon edge servers using ENORM on a cloud server located in the Dublin and N. Virginia data centers respectively. As presented in Section 3.1, there are three stages, namely handshaking during which the edge node is made available for hosting the partitioned server, deployment during which the application server is deployed on the edge node container, and termination during which control from the edge node is given back to the cloud server. We observed the overhead for each stage.

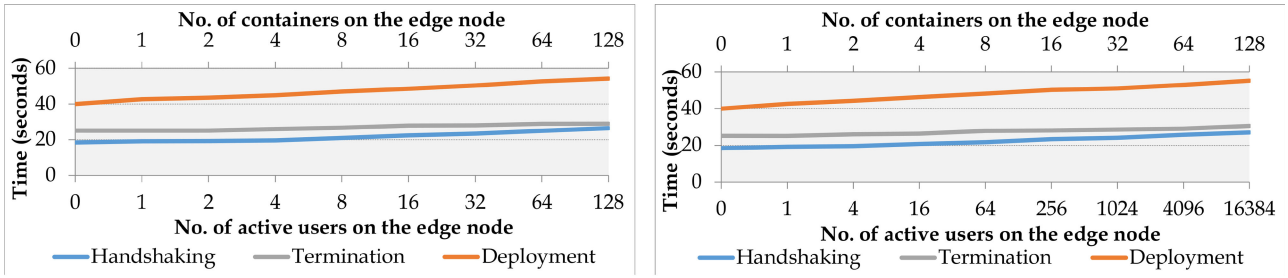
The evaluation takes two scenarios into account. First, when only one user is connected to each server (Figs. 5a and 6a) and when multiple users are connected to each server (Figs. 5b and 6b). Given $n = 0, 1, 2, \ldots, 128$ containers executing on the edge node, we obtained the handshaking overhead when the cloud server needs to establish connection with a new container on the edge node. In this paper, we define handshaking overhead as the time taken for identifying an edge node for deploying an application, initialising a container on the edge node and setting up appropriate firewalls. Similarly, the deployment overhead when a new application server is deployed on the edge node and the termination overhead when a container is terminated on the edge node are obtained. The general trend is that there is a slight increase in the overheads when there is more stress on the edge node. This is expected, but it is worthwhile to note that there is less than a 3 percent increase in the overheads when multiple users connect to the edge servers in comparison to when single users are connected (compare Figs. 5a and 5b). This overhead is because the users' information needs to be transferred from the cloud to the edge during deployment and from the edge to the cloud during termination, which is a single state information, rather than the intermediate states which are maintained on the edge node. This highlights that ENORM can handle a large number of user connections and edge servers for the iPokeMon use-case.

The overheads when using the cloud server in the N. Virginia data center is higher than in Dublin because the handshaking requires more time to communicate given the geographic distance (similarly for termination and deployment). The deployment overhead is under 1 minute in all cases. Until the edge node server is deployed and fully running, the users are still connected to the cloud server and user requests are serviced by the cloud. A user is redirected to the edge node only after the server is deployed.

The deployment overhead is higher than the handshaking and termination overheads. This is because in ENORM, each time a new server is instantiated using a container on the edge node, new packages relevant to the application are installed in the container to host the partitioned server. A custom image of the container hosting the partitioned server is not created by the cloud manager, instead a lean container

12. http://crawdad.org/ctu/personal/20120315/

Authorized licensed use limited to: International Institute of Information Technology-Raipur. Downloaded on January 30,2026 at 10:44:11 UTC from IEEE Xplore. Restrictions apply.



(a) For a single user on each container

(b) For multiple users on each container

Fig. 6. Overheads in provisioning edge servers using ENORM on an Amazon EC2 server located in the N. Virginia data center.

with a basic image suited for an edge node processor architecture is used. For example, an Alpine Linux image suited for the ARM architecture is used on the Odroid board, but a different image would be required for an Intel architecture edge node. Any additional packages specific to an application needs to be installed by the cloud manager during deployment. Given that edge nodes are less likely to be uniform environments with homogeneous processors, ENORM assumes heterogeneity of edge nodes and installs containers suited to the edge node processor architecture. A pre-built image specific to an application may be used by the cloud manager, but requires knowledge of all processor architectures that will be employed at the edge.

Auto-scaling: For evaluating the overhead associated with auto-scaling we considered the following two scenarios. First, a single user per container (only one user is connected to an edge server, up to a maximum of 128 users on the edge node). Second, multiple users per container (given N edge servers, N users are connected per server, up to a maximum of 16,384 users on the edge node). In both scenarios, the experiment started with N containers. For auto-scaling there are three considerations: (i) a container needs to scale up given that it has a high priority and needs to improve the service it offers, (ii) a container needs to scale down, but its application latency will not be affected, and (iii) a container needs to be terminated on the edge node given its lower priority. Using the ENORM framework Procedure 5 is executed by the auto-scaler. Consequently, containers with a higher priority scale up, a number of containers scale down since removing resources do not affect the service it offers and containers are terminated.

Fig. 7 shows the total time taken to execute the auto-scaling mechanism over N containers in the system. When the edge server has only one container there is an overhead of 5.3 seconds. However, when more containers are executed on the system, there is additional overhead (compare bars of the same color) in monitoring all the containers. Increasing the number of users in a container also affects the overhead (compare yellow and green bars) because more users have to be monitored. The auto-scaling overhead impacts decision-making on how frequently auto-scaling can be performed on an edge node for a given workload. In this case, for 128 containers there is an overhead of up to 11 seconds. It would not be worthwhile to auto-scale, for example every 30 seconds or 1 minute, which will render the system unstable, but may be appropriate for larger time intervals to update changing priorities of applications.

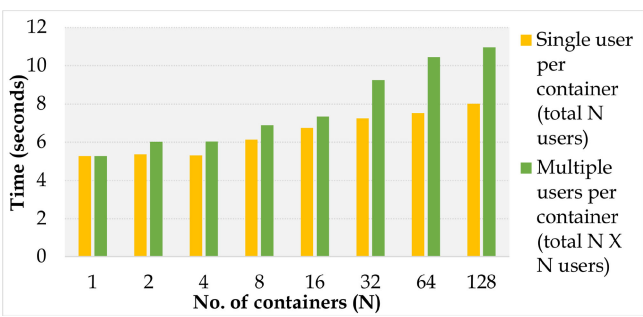


Fig. 7. Total overhead for auto-scaling N containers on the edge node.

5.2 Service Improvement Using ENORM

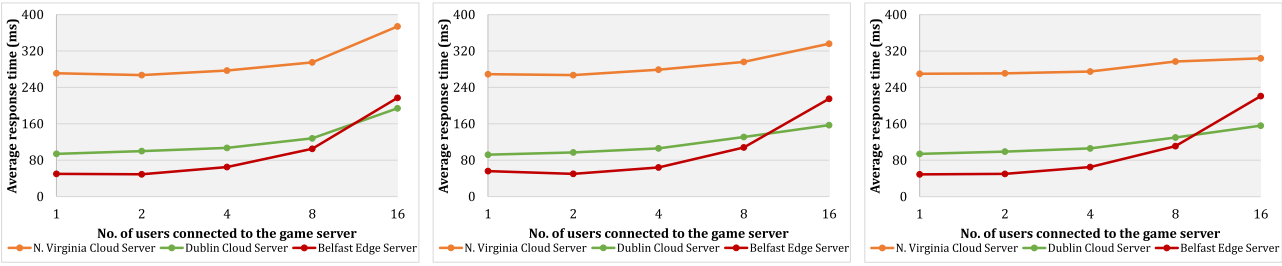
The benefit of using ENORM to achieve the fog computing based iPokeMon is highlighted in terms of reduced application latency, reduced data transfer between a user device and the cloud server and reduced frequency of communication between a user device and the cloud server.

Application latency: To evaluate application latency, both aggressive and mixed user behaviours are considered. Aggressive user behaviour demonstrates a bandwidth-hungry task. Fig. 8 shows the application latency for iPokeMon with aggressive user behaviour when using the ENORM framework in a single server with multiple users scenario. Using JMeter we considered three network configurations: (i) 3G network; refer Fig. 8a, (ii) 4G network; refer Fig. 8b, and (iii) Wi-Fi network; refer Fig. 8c. All three network configurations seem to have little influence on the application latency. This is because we did not exhaust the bandwidth in every HTTP request. The results may be different for other use cases that will exhaust the bandwidth. It is found that the edge server performs better than the cloud servers until there are more than 8 concurrent users. The application overhead increases beyond 8 users. Therefore, in this context for bandwidth-hungry tasks for iPokeMon with more than 8 aggressive users and a total of 73.3 requests per second it is not useful to offload on to an edge server.

To demonstrate a real world setting, mixed user behaviour is considered where the collection of user requests includes the transfer of small and large data and accounts for pauses between requests. Since the network configuration had little influence on application latency, we present all further results with 4G network configuration.

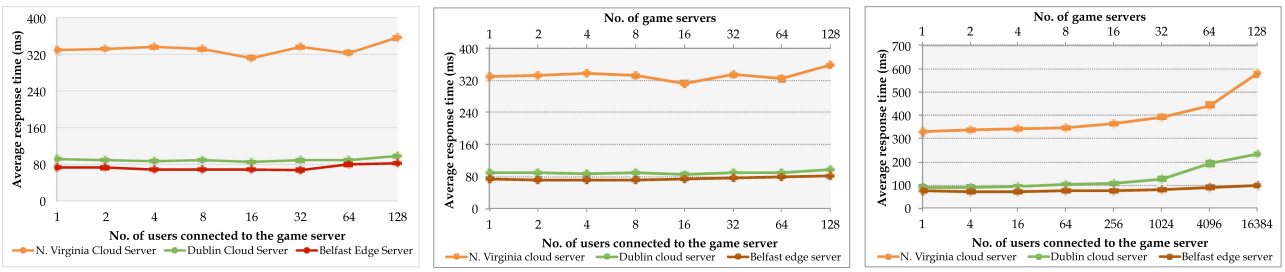
Fig. 9 shows the reduction in application latency for iPokeMon with mixed user behaviour when employing ENORM. Three cases are considered: (i) using a single server with multiple users; refer Fig. 9a, (ii) using multiple servers

Authorized licensed use limited to: International Institute of Information Technology-Raipur. Downloaded on January 30,2026 at 10:44:11 UTC from IEEE Xplore. Restrictions apply.



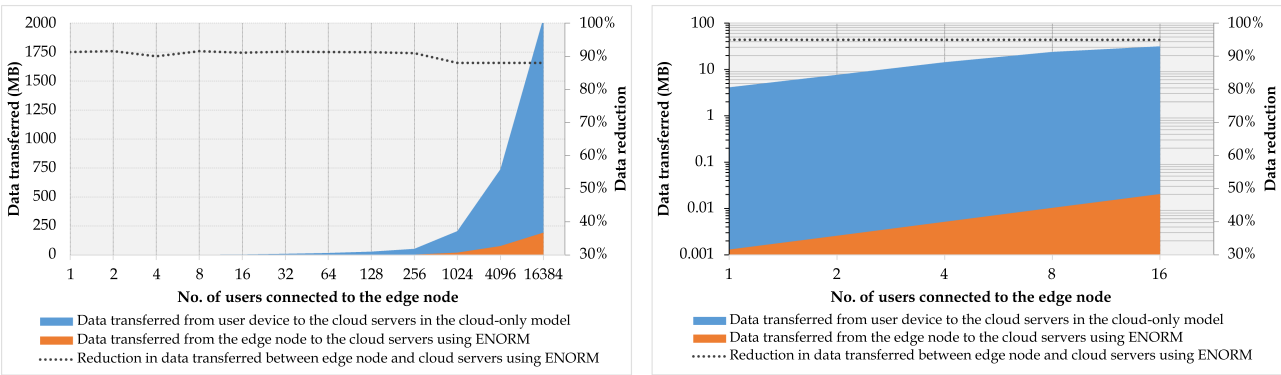
(a) 3G network (b) 4G network (c) Wi-Fi network

Fig. 8. Comparing application latency in the cloud-only and fog computing based iPokeMon game with aggressive user behaviour.



(a) Single server, multiple users (b) Multiple servers, single user (c) Multiple servers, multiple users

Fig. 9. Comparing application latency in the cloud-only and fog computing based iPokeMon game with mixed user behaviour.



(a) Mixed user behaviour (b) Aggressive user behaviour

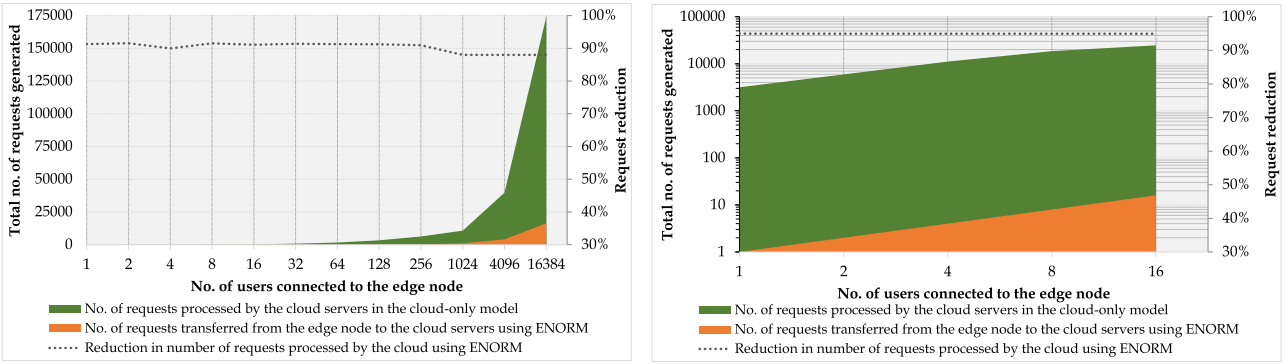
Fig. 10. Comparing the amount of data transferred in the cloud-only and fog computing based iPokeMon game.

with a single user; refer Fig. 9b, and (iii) using multiple servers with multiple users; refer Fig. 9c. For comparison we employed two cloud servers, the first is located in Dublin, which is closest to the location of our edge node and the second is located in N. Virginia. In all three cases, it is immediately inferred that despite the increasing number of servers or users there is a reduction in the application latency. In the cases of a single server (multiple users) and multiple servers (single user), approximately 20 percent reduction is noted in the application latency when using the fog computing model and comparing against the Dublin server. The latency for the application further decreases by nearly 80 percent if the N. Virginia cloud server is used (this is because N. Virginia is geographically further from Dublin).

Fig. 9c shows a more realistic case when there are multiple application servers serving multiple users. In this case, it is observed that for 16,384 users a single request is serviced in 600 ms by the N. Virginia cloud server. On the other hand, the edge server can furnish this request in less than 100 ms. This is a 83 percent improvement. Similarly, the application latency is reduced for the Dublin server.

Data transfer: Fig. 10 shows the reduction in the amount of data transferred between an edge node and the cloud server in the iPokeMon use-case for mixed user behaviour (Fig. 10a) and aggressive user behaviour (Fig. 10b). It is observed that using ENORM the partitioned game server can process the data generated by user devices. On average the data transferred between the edge node and the cloud server is reduced by over 88 percent for mixed user behaviour and up to 95 percent is achieved for aggressive user behaviour (although the edge server is not scalable) in our use-case. This is encouraging in the context of a large volume of data generating devices that will be connected to the Internet. ENORM can facilitate computing closer to the device at the edge node layer such that very little traffic is generated beyond the first hop of the network. For example, when 16,384 users connect to the iPokeMon server from the same location for 5 minutes, 2,000 MB of data is generated. In the cloud-only model all data is sent to the cloud server, but using ENORM only 190 MB of data is sent to the cloud beyond the edge node. This corresponds to approximately 12 KB data per user (190 MB/16,384 users) transferred

Authorized licensed use limited to: International Institute of Information Technology-Raipur. Downloaded on January 30,2026 at 10:44:11 UTC from IEEE Xplore. Restrictions apply.



(a) Mixed user behaviour (b) Aggressive user behaviour

Fig. 11. Comparing the communication frequency in the cloud-only and fog computing based iPokeMon game.

between the edge node and cloud during the redirection from the cloud server to the edge server, which includes a configuration file containing the dynamic IP address of the edge node and the data of the user in the local view.

Communication Frequency: Fig. 11 shows the reduced frequency of communication between an edge node and the cloud server in the iPokeMon use-case for mixed user behaviour (Fig. 11a) and aggressive user behaviour (Fig. 11b). This is similar to data transfer shown in Fig. 10. The number of requests generated in a 5-minute interval for varying number of active users is reduced by 88-95 percent using ENORM when compared to the cloud-only model. For example, when 16,384 active users connect to the iPokeMon server from the same location for 5 minutes, nearly 175,000 requests are sent to the cloud server in the cloud-only model. Using ENORM, the edge node can service over 90 percent requests and only forwards less than 10 percent requests to the cloud server in the fog computing model. Again the benefit of ENORM in the fog computing model is obvious in reducing the number of user requests that needs to be serviced by the cloud. Only between 8-12 percent of requests need to be forwarded from the edge to the cloud server.

5.3 Summary

We summarise the experimental evaluation as follows. First, there are provisioning (handshaking, deployment and termination) and auto-scaling overheads when using ENORM. It is noted for the use-case that there is only a small increase in the overheads; the overhead for handshaking (9 seconds), deployment (9 seconds), termination (5 seconds) and auto-scaling (6 seconds) when moving from one user to 16,384 users in the edge node.

Second, ENORM improves the QoS of iPokeMon. This is observed by reducing the application latency up to a maximum of 95 percent in the best case and reducing data transfer and communication frequency between the edge node and the cloud both by up to 95 percent.

6 Related Work

There are numerous challenges in managing resources in a distributed computing environment. Mechanisms to address these challenges have been developed and explored for different environments, such as grids [7], clusters [4], and more recently on the cloud [5].

The key challenges addressed in relation to managing cloud resources, for example, include (i) scheduling for efficiently mapping workloads on to computing resource [8], (ii) benchmarking for selecting computing resources most suitable for a workload at a given time [9], (iii) monitoring for tracking performance of cloud resources against service agreements and user objectives [10], and (iv) resource scaling for adding or removing cloud resources to meet the computational and storage demands of a workload [11].

With the possibility of extended cloud environments, as envisioned in fog computing that integrates a large volume of limited hardware resource edge nodes, the above challenges will need to be addressed in this new context [2], [3]. The techniques that are employed for cloud resource management are scalable in a data center and even multiple data centers, but assume the concentration of resources. Edge specific characteristics, such as offering a service from a resource distant from the cloud, but closer to a user device, and computing on resource limited nodes will need to be accounted for. Although resource negotiators [12], [13] have been proved efficient in distributed clusters, they are specifically designed for large jobs, such as Hadoop or MapReduce and merely focus on pre-deployment resource provisioning. Simply applying existing cloud-based techniques in the edge context will not be feasible since resource management on the edge will need to be lightweight (consuming minimal resources) and able to dynamically manage resources after deployment in order to support multiple tenants in resource deprived environments. There is limited research exploring lightweight resource management techniques on the cloud [14] that could be directly applied in the context of the edge. However, further research in the edge context will be required to efficiently manage edge resources. In this paper, we set out to investigate techniques suited for the fog computing model, such that the QoS can be improved.

Reference architectures for edge/fog computing have recently been released [15], [16], which highlight the necessary functional layers. These provide a high-level description instead of an implementation. However, in this paper we aim to realise our proposed resource management architecture by implementing ENORM for a real world application.

In the fog computing model, edge nodes are currently used in the following four ways to execute workloads. First,

Authorized licensed use limited to: International Institute of Information Technology-Raipur. Downloaded on January 30,2026 at 10:44:11 UTC from IEEE Xplore. Restrictions apply.

in an aggregating model, in which data from multiple devices are collected by an edge node for pre-processing and filtering [17] tasks. This model is conducive in sensor-based environments, such as wireless sensor networks [18] and the upcoming Internet-of-Things (IoT) [19]. An application gathers data from multiple devices or sensors and filters them on the edge node without routing the entire data to the cloud on which the application resides [20].

Second, a sharing model is employed in which user devices, such as smartphones, tablets and laptops, that volunteer in a mobile cloud offer their spare computational cycles for executing a peer's workload [21]. This model has been employed in video transcribing and face recognition use-cases [22]. The obvious disadvantage of this model is that it can only cover devices in a small region such as a shopping mall [23] or buses/trains [24] and is known for unstable computing due to the mobility of devices [25].

Third, an offloading model can be employed in which workloads can be transferred from a user device to an edge node [26]. There is research highlighting the benefit of such an offloading model for workloads requiring numerical operations [27], for face recognition applications [28] and online games [29]. This is usually done in the context of a mobile edge cloud [30] or cloudlets [31].

The above three models cannot fully leverage the benefit of the edge for improving the QoS of a cloud application. This is because in the aggregating model, the server resides on the cloud and the edge node is only employed for pre-processing data which does not reduce the round-trip latency for a user. In the sharing model, given that peer nodes owned by individual users may not be able to offer continuous edge services, they cannot be employed to host application servers. In the offloading model from the user device to the edge, there is a limited case for complementing the computing requirements of user devices. Applications that are hosted as services require frequent communication between multiple users and the server. To improve the QoS, the frequency of communication between the users and the cloud will need to be minimised to improve the response time. This will be possible if computing is closer to the user, which is not supported in the above models. Hence, in this paper, we consider an alternate execution model of offloading workloads from the cloud server to an edge node.

Current research in offloading workloads from a cloud server to an edge node focuses on caching [32], context-aware web browsing [33] and video pre-processing [34]. There is minimal research addressing resource management. Therefore, in this paper, we set out to address the three challenging problems in resource management when workloads are offloaded from the cloud to the edge. The first problem is partitioning a cloud workload for an edge node. In the context of edge nodes, there is limited research on partitioning servers suitably [35], instead of deploying duplicate servers [36]. The second problem is provisioning edge nodes and facilitating communication between the edge and the cloud servers. Existing edge-based research focuses on harnessing cloudlets without leveraging computing on nodes between the cloud and the user device along the data path [37]. FocusStack is recently proposed for the discovery of edge nodes using geographic addressing in order to assist workload deployments [38]. Our paper on the other hand take the next step after discovery for managing resources by assuming that they have already been discovered using such approaches. Additionally, we consider a multi-tenant environment that is not considered in FocusStack. ParaDrop is designed to support multi-tenancy in Wi-Fi Access Points, with a cloud-based management to orchestrate applications across multiple edge nodes [39]. However, it only considers using the edge nodes as a replacement to clouds, which is fundamentally different to our approach in which both the cloud and edge are employed for improving the QoS of an application. Additionally, we have tested our approach on a real application. The third problem is dynamic management of resources on the edge to meet service objectives. There is auto-scaling research in the context of cloud VMs [40], but cannot be directly applied to the edge. Our research addresses this challenge in the context of containers. Techniques for auto resource provisioning and on-demand consumption of edge resources are proposed for IoT applications [41] but do not support post-deployment resource management for multi-tenant environment.

7 Conclusions

As more and more devices get added to the Internet, computing on nodes, such as routers, base stations and switches, at the edge of the network will need to be tapped into. This is the vision of fog computing. However, to realise this vision a number of problems will need to be solved. In this paper, we focused on the problems related to resource management. Existing resource management frameworks in distributed computing are suitable in the context of clouds and clusters, but do not integrate the edge of the network.

In this paper we have presented ENORM, an *E*dge *NO*de *R*esource *M*anagement framework that integrates edge nodes in the computing ecosystem to realise fog computing. ENORM addresses the resource management problems of provisioning edge nodes for cloud applications, deploying workloads on provisioned edge nodes, and dynamic resource allocation on edge nodes. To this end, we proposed a provisioning mechanism that considers handshaking, deployment of workloads and termination of edge services. Additionally, an auto-scaling mechanism to dynamically manage edge resources is developed. The provisioning and auto-scaling mechanisms are simple implementations based on linear search algorithm given that edge nodes are resource constrained environments.

The feasibility of ENORM was tested on a PokéMon Go-like online game use-case. Experimentally it was noted that there are significant benefits in improving the QoS for a large number of users in a given location for the fog computing based use-case employing ENORM. When compared to a cloud-only model, the application latency is reduced between 20-80 percent. Similarly, the data traffic and the communication frequency between the edge node and the cloud server are both reduced up to 95 percent.

7.1 Limitations

One shortcoming of the auto-scaler we have proposed is that it only considers static priorities of applications that are set by the cloud manager instead of dynamic priorities

Authorized licensed use limited to: International Institute of Information Technology-Raipur. Downloaded on January 30,2026 at 10:44:11 UTC from IEEE Xplore. Restrictions apply.

which are more realistic in real-world settings. The priority of an application may need to change when more users subscribe to the application. This will need to be considered with business models that make edge resources publicly available. Immediate efforts will be made to investigate this.

The benefit of using fog computing for massively geo-distributed applications across data centers may not be obvious since the reduction in latency may not be significant to motivate the use of the edge of the network. However, the number of data centers is less likely to grow at the same rate as the number of devices since they consume lots of power and global network bandwidth. The edge of the network may be used to improve the QoS of applications.

7.2 Future Work

Currently, ENORM is suited for both single and multiple edge nodes environment. However, migrating applications between nodes is not considered in this paper. This will require synchronisation between edge nodes and suitable techniques for migrating or handing over a service onto another node. This will be investigated in the future.

The current auto-scaling mechanism employed in ENORM adds a unit of both CPU and memory resources since it reduces the application latency of our use case. Other use cases may need to consider more resources (for example I/O) or a different combination of resources. In addition, ENORM's performance may be affected by the granularity of the unit value. Our hypothesis is that when the unit value is highly fine-grained, the effect on the QoS of an application by scaling resources may be less obvious. A more coarse-grained value could result in an over provision resources required by the container. The optimal value for the unit value given an application may vary and will need to be explored through experiments. Determining a dynamic value by using heuristics may further optimise a static value that is used throughout the execution of an application. This will be investigated in the future.

Acknowledgments

The authors acknowledge the SFI-DEL grant (14/IA/2474).

References

[1] P. Garcia Lopez, et al., "Edge-centric computing: Vision and challenges," *SIGCOMM Comput. Commun. Rev.*, vol. 45, no. 5, pp. 37–42, 2015.

[2] A. V. Dastjerdi and R. Buyya, "Fog computing: Helping the internet of things realise its potential," *Comput.*, vol. 49, no. 8, pp. 112–116, 2016.

[3] B. Varghese, N. Wang, S. Barbhuiya, P. Kilpatrick, and D. S. Nikolopoulos, "Challenges and opportunities in edge computing," in *Proc. IEEE Int. Conf. Smart Cloud*, 2016, pp. 20–26.

[4] J. Polo, et al., "Resource-aware adaptive scheduling for MapReduce clusters," in *Proc. ACM/IFIP/USENIX Int. Conf. Middleware*, 2011, pp. 187–207.

[5] B. Jennings and R. Stadler, "Resource management in clouds: Survey and research challenges," *J. Netw. Syst. Manage.*, vol. 23, no. 3, pp. 567–619, 2015.

[6] W. Felter, A. Ferreira, R. Rajamony, and J. Rubio, "An updated performance comparison of virtual machines and Linux Containers," in *Proc. IEEE Int. Symp. Performance Anal. Syst. Softw.*, 2015, pp. 171–172.

[7] R. Buyya, D. Abramson, J. Giddy, and H. Stockinger, "Economic models for resource management and scheduling in Grid computing," *Concurrency Comput.: Practice Experience*, vol. 14, no. 13–15, pp. 1507–1542, 2002.

[8] H. Chen, X. Zhu, D. Qiu, and L. Liu, "Uncertainty-aware real-time Workflow scheduling in the cloud," in *Proc. IEEE Int. Conf. Cloud Comput.*, 2016, pp. 577–584.

[9] B. Varghese, O. Akgun, I. Miguel, L. Thai, and A. Barker, "Cloud benchmarking for maximising performance of scientific applications," *IEEE Trans. Cloud Comput.*, 2016.

[10] J. Montes, A. Sánchez, B. Memishi, M. S. Pérez, and G. Antoniu, "GMonE: A complete approach to cloud monitoring," *Future Generation Comput. Syst.*, vol. 29, no. 8, pp. 2026–2040, 2013.

[11] R. Han, L. Guo, M. M. Ghanem, and Y. Guo, "Lightweight resource scaling for cloud applications," in *Proc. IEEE/ACM Int. Symp. Cluster Cloud Grid Comput.*, 2012, pp. 644–651.

[12] V. K. Vavilapalli, et al., "Apache hadoop yarn: Yet another resource negotiator," in *Proc. Symp. Cloud Comput.*, 2013, pp. 5:1–5:16.

[13] B. Hindman, et al., "Mesos: A platform for fine-grained resource sharing in the data center," in *Proc. USENIX Symp. Netw. Syst. Des. Implementation*, 2011, p. 22.

[14] B. Varghese, L. Subba, L. Thai, and A. Barker, "DocLite: A docker-based lightweight cloud benchmarking tool," in *Proc. IEEE/ACM Int. Symp. Cluster Cloud Grid Comput.*, 2016, pp. 213–222.

[15] Mobile Edge Computing (MEC); Framework and Reference Architecture, (Date last accessed 01-May-2017). [Online]. Available: http://www.etsi.org/deliver/etsi_gs/MEC/001_099/003/01.01.01_60/gs_MEC003v010101p.pdf

[16] OpenFog Reference Architecture, (Date last accessed 01-May-2017). [Online]. Available: https://www.openfogconsortium.org/ra/

[17] Y. Yao, Q. Cao, and A. V. Vasilakos, "EDAL: An energy-efficient, delay-aware, and lifetime-balancing data collection protocol for wireless sensor networks," in *Proc. IEEE Int. Conf. Mobile Ad-Hoc Sensor Syst.*, 2013, pp. 182–190.

[18] S. Roy, M. Conti, S. Setia, and S. Jajodia, "Secure data aggregation in wireless sensor networks: Filtering out the attacker's impact," *IEEE Trans. Inf. Forensics Secur.*, vol. 9, no. 4, pp. 681–694, Apr. 2014.

[19] Z. Zhou, J. Tang, L.-J. Zhang, K. Ning, and Q. Wang, "EGF-tree: An energy-efficient index tree for facilitating multi-region query aggregation in the internet of things," *Personal Ubiquitous Comput.*, vol. 18, no. 4, pp. 951–966, 2014.

[20] L. A. Villas, A. Boukerche, H. A. B. F. De Oliveira, R. B. De Araujo, and A. A. F. Loureiro, "A spatial correlation aware algorithm to perform efficient data collection in wireless sensor networks," *Ad Hoc Netw.*, vol. 12, pp. 69–85, 2014.

[21] A. Zhou, S. Wang, J. Li, Q. Sun, and F. Yang, "Optimal Mobile Device Selection for Mobile Cloud Service Providing," *J. Supercomputing*, vol. 72, no. 8, pp. 3222–3235, 2016.

[22] N. Fernando, S. W. Loke, and W. Rahayu, "Computing with nearby mobile devices: A work sharing algorithm for mobile edge-clouds," *IEEE Trans. Cloud Comput.*, 2016.

[23] I. Farris, L. Militano, M. Nitti, L. Atzori, and A. Iera, "MIFaaS: A Mobile-IoT-Federation-as-a-Service Model for Dynamic Cooperation of IoT Cloud Providers," *Future Generation Comput. Syst.*, vol. 70, pp. 126–137, 2017.

[24] K. Habak, M. Ammar, K. A. Harras, and E. Zegura, "Femto clouds: Leveraging mobile devices to provide cloud service at the edge," in *Proc. IEEE Int. Conf. Cloud Comput.*, 2015, pp. 9–16.

[25] W. Gao, "Opportunistic peer-to-peer mobile cloud computing at the tactical edge," in *Proc. IEEE Military Commun. Conf.*, 2014, pp. 1614–1620.

[26] S. Sardellitti, G. Scutari, and S. Barbarossa, "Joint optimisation of radio and computational resources for multicell mobile-edge computing," *IEEE Trans. Signal Inf. Process. Over Netw.*, vol. 1, no. 2, pp. 89–103, Jun. 2015.

[27] D. G. Roy, D. De, A. Mukherjee, and R. Buyya, "Application-aware cloudlet selection for computation offloading in multi-cloudlet environment," *J. Supercomput.*, vol. 73, pp. 1–19, 2016.

[28] B. Zhou, A. V. Dastjerdi, R. Calheiros, S. Srirama, and R. Buyya, "mCloud: A context-aware offloading framework for heterogeneous mobile cloud," *IEEE Trans. Serv. Comput.*, 2015.

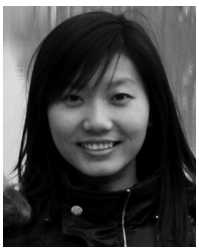
[29] E. Cuervo, et al., "MAUI: Making smartphones last longer with code offload," in *Proc. Int. Conf. Mobile Syst., Appl. Serv.*, 2010, pp. 49–62.

[30] S. Sardellitti, G. Scutari, and S. Barbarossa, "Efficient multi-user computation offloading for mobile-edge cloud computing," *IEEE/ACM Trans. Netw.*, vol. 24, no. 5, pp. 2795–2808, Oct. 2016.

[31] L. Wang, L. Jiao, D. Kliazovich, and P. Bouvry, "Reconciling task assignment and scheduling in mobile edge clouds," in *Proc. Int. Workshop Hot Topics Practical Netw. Syst.*, 2016, pp. 1–6.

Authorized licensed use limited to: International Institute of Information Technology-Raipur. Downloaded on January 30,2026 at 10:44:11 UTC from IEEE Xplore. Restrictions apply.

[32] K. Bhardwaj, P. Agrawal, A. Gavrilovska, and K. Schwan, "AppSachet: Distributed app delivery from the edge cloud," in *Proc. 7th Int. Conf. Mobile Comput. Appl. Serv.*, 2015, pp. 89–106.
[33] P. Savolainen, et al., "Spaceify: A client-edge-server ecosystem for mobile computing in smart spaces," in *Proc. Int. Conf. Mobile Comput. Netw.*, 2013, pp. 211–214.
[34] P. Simoens, Y. Xiao, P. Pillai, Z. Chen, K. Ha, and M. Satyanarayanan, "Scalable crowd-sourcing of video from mobile devices," School of Computer Science, Carnegie Mellon University, Tech. Rep. CMU-CS-12–147, 2012. [Online]. Available: http://elijah.cs.cmu.edu/DOCS/CMU-CS-12-147.pdf
[35] B.-G. Chun, S. Ihm, P. Maniatis, M. Naik, and A. Patti, "Clonecloud: Elastic execution between mobile device and cloud," in *Proc. Conf. Comput. syst.*, 2011, pp. 301–314.
[36] M. Baguena, G. Samaras, A. Pamboris, M. L. Sichitiu, P. R. Pietzuch, and P. Manzoni, "Towards enabling hyper-responsive mobile apps through network edge assistance," in *Proc. IEEE Annu. Consumer Commun. Netw. Conf.*, 2016, pp. 399–404.
[37] Z. Chen, "Early implementation experience with wearable cognitive assistance applications," in *Proc. Workshop Wearable Syst. Appl.*, 2015, pp. 33–38.
[38] B. Amento, B. Balasubramanian, R. J. Hall, K. Joshi, G. Jung, and K. H. Purdy, "FocusStack: Orchestrating edge clouds using location-based focus of attention," in *Proc. IEEE/ACM Symp. Edge Comput.*, 2016, pp. 179–191.
[39] P. Liu, D. Willis, and S. Banerjee, "ParaDrop: Enabling lightweight multi-tenancy at the networks extreme edge," in *Proc. IEEE/ACM Symp. Edge Comput.*, 2016, pp. 1–13.
[40] S. F. Piraghaj, A. V. Dastjerdi, R. N. Calheiros, and R. Buyya, "Efficient virtual machine sizing for hosting containers as a service," in *Proc. IEEE World Congr. Serv.*, 2015, pp. 31–38.
[41] S. Nastic, H.-L. Truong, and S. Dustdar, "A middleware infrastructure for utility-based provisioning of IoT cloud systems," in *Proc. IEEE/ACM Symp. Edge Comput.*, 2016, pp. 28–40.



Nan Wang received the undergraduate degree from Beijing Jiaotong University, China, the MSc degree in management and information technology from the University of St Andrews, in 2014, and the MRes degree in web science and big data analytics from the University College London, in 2015. She is working toward the PhD degree in computer science at Queen's University Belfast, United Kingdom. Her research interest is in fog computing. More information is available from http://nwang03.public.cs.qub.ac.uk.

Blesson Varghese received the MSc degree in network centred computing and the PhD degree in computer science, both from the University of Reading, United Kingdom, in 2008 and 2011, respectively, on international scholarships. He is a lecturer in computer science with Queen's University Belfast and an honorary lecturer with the University of St Andrews. His interests include developing and analysing novel parallel and distributed systems that leverage the edge of the network. More information is available from www.blessonv.com.

Michail Matthaiou received the MSc degree in communication systems and signal processing from the University of Bristol, United Kingdom, in 2005, and PhD degree from the University of Edinburgh, United Kingdom, in 2008. He is a senior lecturer in the ECIT Institute, Queen's University Belfast, United Kingdom. His interests include signal processing for wireless communications, energy-efficient dense networks, and fog computing.

Dimitrios S. Nikolopoulos is professor and head of the School of Electronics, Electrical Engineering and Computer Science, Queen's University of Belfast. He holds the chair in High Performance and Distributed Computing. His research explores scalable computing systems for data-driven applications and new computing paradigms at the limits of performance, power and reliability. More information is available from http://www.cs.qub.ac.uk/∼D.Nikolopoulos/.

Authorized licensed use limited to: International Institute of Information Technology-Raipur. Downloaded on January 30,2026 at 10:44:11 UTC from IEEE Xplore. Restrictions apply.